

STM32 and MATLAB-Based Multi-Sensor Fusion for Unmanned Aerial Vehicles: 9-Axis Orientation and Altitude Estimation Using a Kalman Filter

ABSTRACT

In this report, multi-sensor data fusion and orientation estimation, which are of critical importance for unmanned aerial vehicle (UAV) flight dynamics and navigation systems, were carried out. Within the scope of the project, raw data were read from MPU6050, QMC5883P, and BMP280 sensors using the STM32F072B Discovery microcontroller board. While sensor readings were performed at the hardware level, they were integrated into the system by taking open-source libraries as references. The obtained data were transferred to the computer environment via the UART protocol at a baud rate of 115200 and an approximate sampling frequency of 1.61 Hz, and were recorded in CSV format using the Tera Term terminal software.

In the second stage, these recorded noisy and drift-prone sensor data were processed with a Kalman filter algorithm developed in the MATLAB environment. The developed algorithm eliminated accelerometer noise, gyroscope drift, magnetometer errors, and instantaneous vibrations in the barometer, and stably estimated the Roll, Pitch, Yaw angles and the Z-axis altitude change.

I. INTRODUCTION

In autonomous systems and unmanned aerial vehicles, in order to ensure stable flight control, the spatial orientation and altitude of the vehicle must be known instantaneously with minimum error. The states of a system are variables that provide a complete representation of the internal condition or status of that system at a particular moment. Today, microelectromechanical system-based inertial measurement units and digital barometric sensors are generally used to obtain these parameters. However, the raw data produced by these sensors are not suitable for use in system control on their own due to motor vibration-induced noise, magnetic interference, and numerical integration errors that accumulate over time. In this study, an end-to-end architecture from hardware to software was designed. Sensor data read on the STM32F072B Discovery hardware were collected via serial communication and analyzed offline in the MATLAB environment. In order to compensate for the weaknesses of the sensors with each other's strengths and to achieve data fusion, a Kalman filter, accepted as a fundamental tool for analyzing and solving various estimation problems, was applied. The overall architecture and workflow of the project are summarized in Figure 1.

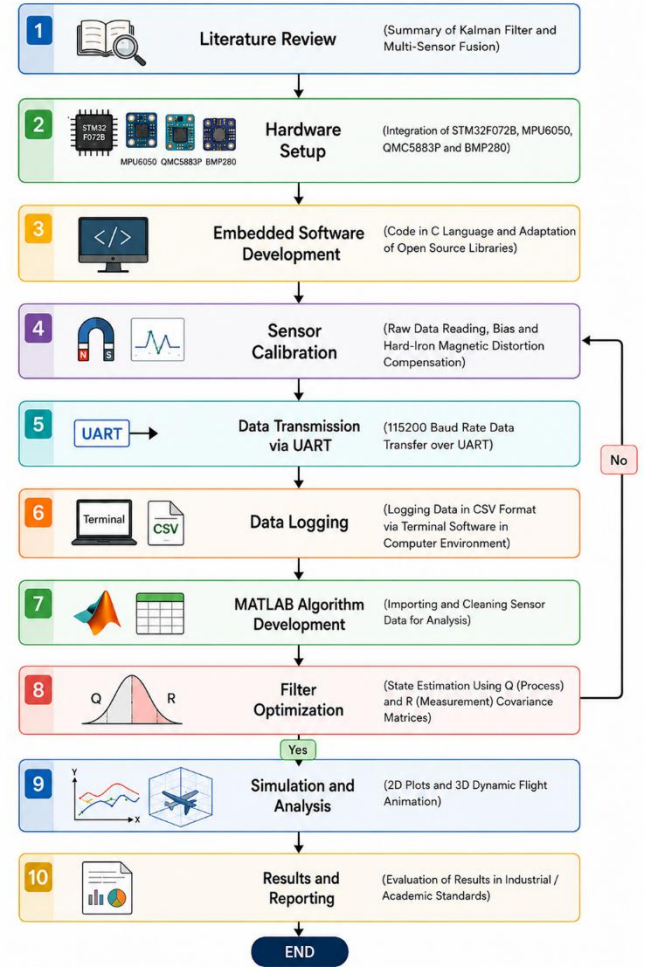


Figure 1. Overall system architecture and data flow diagram of the project. (The visual design was created with the help of artificial intelligence tools.)

II. HARDWARE AND SOFTWARE INFRASTRUCTURE

A. Microcontroller and Sensors

In the project, the STM32F072B Discovery development board was used to perform data acquisition and communication management. Three different environmental sensors were integrated into the system to obtain flight dynamics (Roll, Pitch, Yaw, and Altitude) (Figure 2):

- **MPU6050 (Accelerometer and Gyroscope):** It is a chip-integrated sensor containing a 3-axis MEMS gyroscope and a 3-axis MEMS accelerometer. It has separate 16-bit analog-to-digital converters (ADC) for digitizing the gyroscope and accelerometer outputs.
- **QMC5883P (Magnetometer):** It was used to determine the heading/yaw orientation of the system by measuring the Earth's magnetic field on three axes. It communicates over I2C with the address 0x2C and provides absolute direction determination for the system.

- **BMP280 (Barometer):** It is an absolute barometric pressure sensor designed especially for mobile applications, providing altitude change on the Z-axis by measuring ambient barometric air pressure.

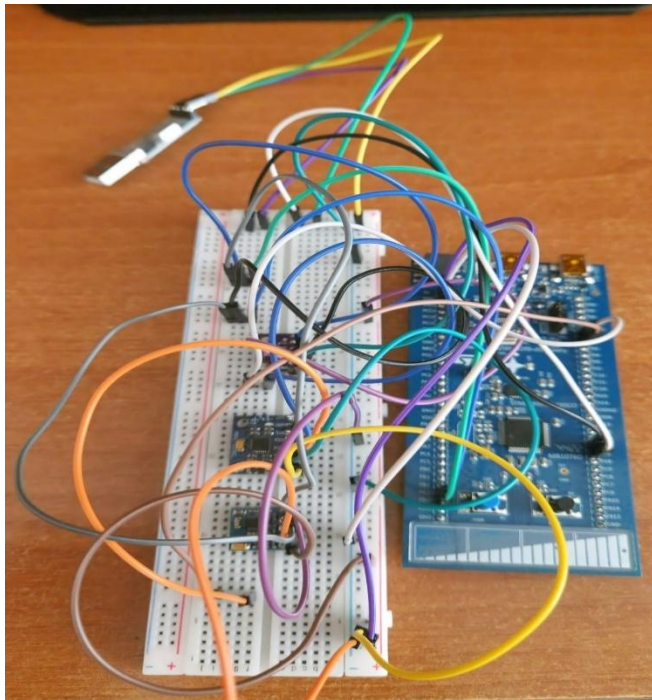


Figure 2. General view of the STM32F072B Discovery board and sensor connections.

B. Development Environments

The embedded software was prepared on the STM32 platform using the C language, while the processing of data obtained from the sensors, the application of mathematical algorithms, and the creation of graphical simulations were carried out in the MATLAB environment. The pin assignments of the peripheral units on the microcontroller were configured through the STM32CubeIDE interface as shown in Figure 3.

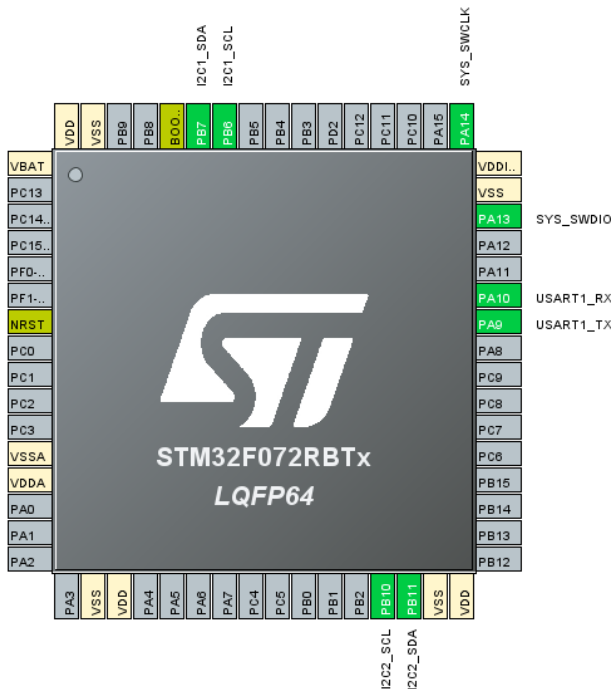


Figure 3. Configuration of peripheral units (I2C and UART) in the STM32CubeIDE environment..

III. DATA ACQUISITION AND COMMUNICATION

The raw values read from the sensors were converted into a digital packet on the microcontroller at a frequency of 1.61 Hz. These data packets were transmitted over the serial port using the Universal Asynchronous Receiver-Transmitter (UART) protocol at a speed of 115200 Baud Rate to be transferred to the computer environment. Tera Term software was used on the computer side to capture the serial data stream (Figure 4).

-0.09,-0.07,1.00,12.27,-1.53,0.87,1817.00,-811.00,-112.00,100508.01
-0.09,0.07,1.01,13.40,4.76,2.15,1826.00,-608.00,-72.00,100510.62
-0.07,0.22,0.96,9.95,-0.02,0.28,1823.00,-374.00,-38.00,100514.55
-0.11,0.34,0.94,8.82,1.73,-3.04,1805.00,-243.00,8.00,100506.27
-0.08,0.38,0.91,1.38,-1.03,1.82,1833.00,-191.00,64.00,100509.31
-0.08,0.42,0.92,-2.03,5.29,-1.38,1824.00,-151.00,69.00,100512.82
-0.07,0.39,0.92,-2.63,3.79,-1.40,1827.00,-168.00,74.00,100511.28
-0.06,0.33,0.96,-11.56,4.16,1.53,1860.00,-267.00,44.00,100512.82
-0.05,0.25,0.95,-12.61,-1.53,0.44,1862.00,-357.00,8.00,100509.12
-0.02,0.11,0.98,-15.15,3.18,1.41,1883.00,-557.00,-40.00,100507.12
-0.08,0.02,0.99,-0.09,24.61,-9.65,1834.00,-689.00,-86.00,100503.00
-0.11,0.00,1.00,-1.67,4.05,7.01,1786.00,-723.00,-112.00,100503.66
-0.12,-0.00,1.01,-2.87,5.18,5.50,1781.00,-757.00,-125.00,100504.97
-0.12,0.02,0.99,-3.19,0.48,23.51,1762.00,-892.00,-130.00,100503.84
-0.14,-0.00,0.99,-3.25,0.36,26.75,1731.00,-1064.00,-116.00,100500.18
-0.11,-0.01,0.99,-3.86,1.24,18.72,1649.00,-1220.00,-120.00,100497.72
-0.14,0.01,0.99,-2.40,1.83,16.53,1580.00,-1324.00,-135.00,100502.30
-0.13,0.00,0.99,-0.57,0.40,5.89,1521.00,-1386.00,-124.00,100498.61
-0.14,0.04,0.98,-3.35,0.84,16.95,1437.00,-1440.00,-132.00,100501.80
-0.10,0.04,0.99,-1.38,1.98,5.19,1347.00,-1476.00,-109.00,100501.80
-0.12,0.01,1.00,-0.93,1.49,0.34,1331.00,-1497.00,-127.00,100503.12
-0.11,0.03,1.00,0.79,1.68,-10.35,1357.00,-1471.00,-102.00,100502.23
-0.13,0.03,0.99,2.63,1.13,-25.12,1473.00,-1411.00,-124.00,100505.45
-0.13,0.01,0.99,2.27,0.71,-23.11,1609.00,-1303.00,-117.00,100499.38
-0.13,0.00,1.00,8.92,-2.27,-27.30,1717.00,-1129.00,-109.00,100500.27
-0.11,0.04,0.99,2.90,0.61,-28.40,1799.00,-933.00,-112.00,100503.50
-0.13,0.01,0.98,6.14,0.76,-23.63,1773.00,-751.00,-133.00,100507.84
-0.13,0.00,0.99,-1.95,-1.11,16.12,1791.00,-819.00,-106.00,100506.95
-0.09,0.01,1.00,-1.63,1.69,7.39,1771.00,-937.00,-116.00,100509.58
-0.10,0.05,1.00,0.53,-0.20,-34.42,1816.00,-642.00,-83.00,100508.69
-0.10,0.00,1.00,1.72,1.71,-31.90,1682.00,-352.00,-93.00,100508.27
-0.09,0.07,0.92,0.35,1.39,-20.03,1531.00,-216.00,-81.00,100505.42
-0.13,0.04,1.00,2.13,5.31,-30.50,1338.00,-96.00,-46.00,100508.50
-0.12,0.04,0.99,0.98,-0.24,-11.40,1200.00,-44.00,-20.00,100509.77
-0.13,0.02,0.99,-0.16,0.24,-2.21,1123.00,-20.00,-19.00,100507.57
-0.12,0.02,1.00,-0.74,-1.39,-9.60,1058.00,-19.00,-4.00,100508.00
-0.10,0.01,1.00,-4.68,0.18,26.59,1115.00,-15.00,-21.00,100509.48
-0.16,0.03,0.98,-4.85,-2.71,44.77,1361.00,-80.00,-47.00,100504.91

Figure 4. Real-time sensor data received via UART in the Tera Term software.

The received data were recorded as a CSV file to be used in offline filtering analyses (Figure 5). The sampling frequency measured as approximately 1.61 Hz resulted from the delay times occurring during real-time logging over UART and is specific to the offline test procedure of the algorithm. Since the purpose of the data acquisition stage was not real-time flight control but verification of the offline performance of the Kalman filter, the low sampling frequency was considered sufficient. In the final flight control application, the filters are planned to be operated at much higher sampling frequencies using the hardware timers of the microcontroller.

	A	B	C	D	E	F	G
1	-0.11,0.00,0.99,-1.11,1.31,-0.24,1784.00,-693.00,-108.00,100505.01						
2	-0.11,0.00,1.01,-0.56,1.07,0.09,1784.00,-693.00,-123.00,100502.02						
3	-0.11,0.01,0.99,-1.03,1.33,-0.05,1783.00,-693.00,-109.00,100505.05						
4	-0.11,0.01,0.99,-0.82,1.13,-0.07,1782.00,-686.00,-112.00,100504.39						
5	-0.11,0.01,0.99,-0.66,1.28,0.09,1779.00,-701.00,-109.00,100500.66						
6	-0.11,0.01,0.99,-0.96,1.08,-0.28,1779.00,-704.00,-123.00,100503.74						
7	-0.11,0.01,0.99,-0.69,1.21,0.07,1781.00,-693.00,-105.00,100505.89						
8	-0.11,0.01,1.00,-0.87,1.36,0.01,1784.00,-692.00,-106.00,100503.74						
9	-0.12,0.01,1.00,-1.01,1.44,0.05,1776.00,-694.00,-122.00,100505.20						
10	-0.12,0.01,1.00,-0.60,1.31,-0.36,1776.00,-690.00,-111.00,100501.55						
11	-0.11,0.01,1.00,-0.95,1.09,0.11,1776.00,-698.00,-116.00,100504.36						
12	-0.11,0.01,1.00,-0.57,1.16,0.11,1780.00,-686.00,-105.00,100505.89						
13	-0.11,0.01,1.00,-0.75,1.29,-0.24,1790.00,-693.00,-136.00,100501.97						
14	-0.11,0.01,0.99,-0.72,1.30,0.02,1789.00,-695.00,-113.00,100503.47						
15	-0.12,0.00,1.00,-0.82,1.11,-0.02,1778.00,-693.00,-119.00,100501.08						
16	-0.11,0.01,1.00,-0.68,1.16,-0.04,1788.00,-695.00,-131.00,100505.89						
17	-0.11,0.01,1.00,-0.72,1.24,-0.08,1783.00,-696.00,-127.00,100501.31						
18	-0.11,0.01,0.99,-0.72,0.95,0.15,1781.00,-696.00,-107.00,100505.43						
19	-0.11,0.01,1.00,-0.91,1.24,-0.18,1776.00,-701.00,-119.00,100503.28						
20	-0.11,0.01,1.00,-0.66,1.08,-0.14,1777.00,-690.00,-121.00,100503.93						
21	-0.11,0.01,0.99,-0.68,1.10,0.04,1781.00,-691.00,-107.00,100503.93						
22	-0.12,0.01,0.99,-0.92,1.34,-0.18,1777.00,-699.00,-121.00,100506.55						
23	-0.13,0.01,1.00,-0.82,1.73,2.36,1751.00,-680.00,-132.00,100505.43						
24	-0.23,0.01,0.97,-0.68,1.10,5.14,1606.00,-687.00,-175.00,100501.73						
25	-0.27,0.01,0.96,-0.89,10.15,0.00,1570.00,-677.00,-187.00,100502.39						
26	-0.35,0.01,0.94,-1.14,1.57,0.18,1469.00,-669.00,-208.00,100504.55						

Figure 5. Sensor data set transferred to the MATLAB environment and used for filtering.

IV. KALMAN FILTER MODELING

The recorded raw data were transferred to the MATLAB environment and Kalman filter algorithms were designed for multi-axis state estimation. The filtering process consists of mathematical cycles in which the prediction and update steps of state estimation proceed sequentially. The standard discrete-time Kalman filter equations forming the mathematical basis of the system and repeated at each sampling step are as follows:

Prediction Step:

$$x_{k|k-1} = Ax_{k-1|k-1} + Bu_k$$

$$P_{k|k-1} = AP_{k-1}A^T + Q$$

Update Step:

$$K = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}$$

$$x_k = x_{k|k-1} + K(z_k - Hx_{k|k-1})$$

$$P_k = (I - KH)P_{k|k-1}$$

Here, x represents the state vector, A the state transition matrix, B the control input matrix, u the control input, P the error covariance matrix, Q the process noise matrix, K the Kalman gain, z the sensor measurement vector, H the measurement matrix, and R the measurement noise covariance matrix.

To establish a balance between the system's response time (aggressiveness) and smoothness, the Process Noise (Q) and Measurement Noise (R) covariance matrices were experimentally adjusted.

- **Roll and Pitch Filtering:** High-frequency signal noise of the accelerometer and low-frequency, time-increasing drift errors of the gyroscope were combined; clear inclination angles were obtained by optimizing the noise covariance matrices.
- **Yaw Filtering:** The absolute compass orientation obtained from the magnetometer and the angular velocity of the gyroscope on the Z-axis were synthesized. Before projecting the magnetometer data onto the horizon plane, the min-max calibration method applied by rotating the system in different directions was used to eliminate hardware and environmental magnetic distortions, and the obtained constant offset values were integrated into the raw data. Then, tilt compensation was applied and the magnetometer data were projected onto the horizon plane using the roll and pitch angles.
- **Altitude (Z-Axis) Filtering:** Altitude data based on air pressure obtained from the barometer is filtered to provide the system with smooth altitude information. Due to the high noise variance in the barometer data, stability is ensured by keeping the process noise (Q) low and the measurement noise (R) high because of the high variance in the barometer measurements, since the mathematical model is relied upon in altitude estimation.

V. SIMULATION AND RESULTS

The success of the Kalman filter was verified by comparative graphs plotted in MATLAB and three-dimensional orientation visualization.

The obtained axis analyses are presented in Figure 6. The noisy structure of the accelerometer data shown in blue and the significant drift of the gyroscope data shown in red were substantially suppressed after filtering, as expressed by the thick green Kalman curve. In the altitude (Z-axis) graph, it was calculated that the high-frequency vibrations in the raw barometer data were reduced by approximately 95% after the application of the Kalman filter, proving that a stable line was achieved. In the data fusion of the Yaw axis (Figure 7), instantaneous breaks in the magnetometer data and the erroneous orientation of the gyroscope were balanced.

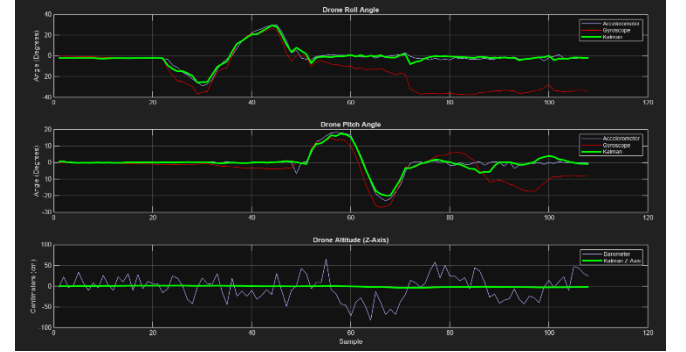


Figure 6. Comparison of the results obtained with the Kalman filter on the Roll, Pitch, and altitude axes..

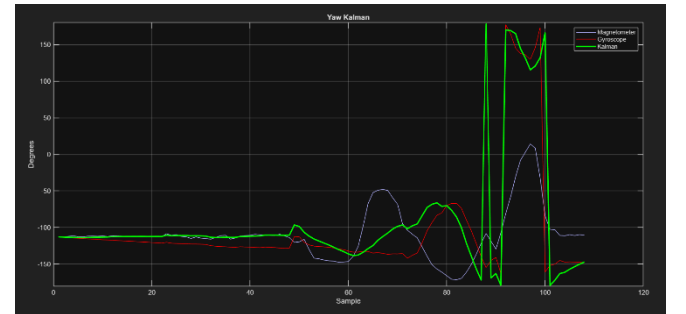


Figure 7. Orientation estimation obtained with the Kalman filter on the Yaw axis.

Finally, three-dimensional orientation visualization was performed using the filtered Euler angles (Figures 8-13). In the visualization, the orientation changes of the 3D model created using the Roll, Pitch, and Yaw angles together with the Z-axis translation were presented in a more stable and readable manner after being purified from sensor noise.

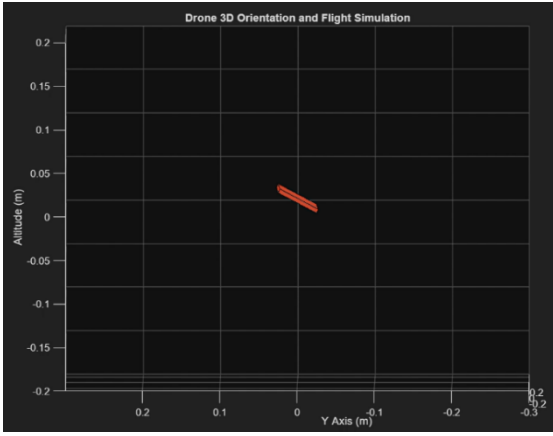


Figure 8. Right roll movement in the three-dimensional simulation.

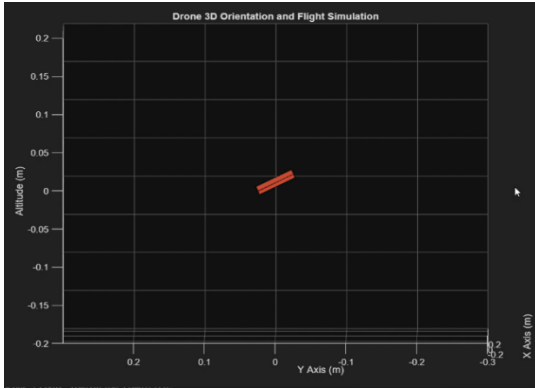


Figure 9. Left roll movement in the three-dimensional simulation

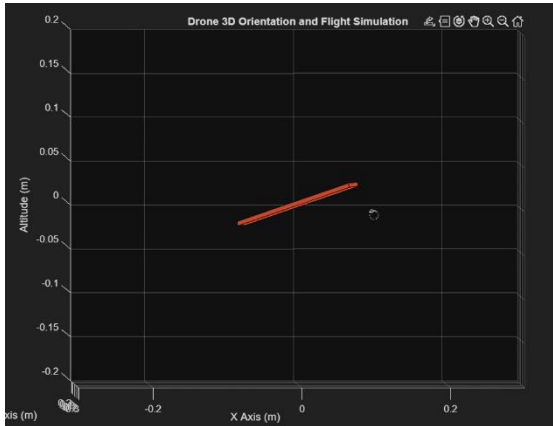


Figure 10 Pitch Up movement in the three-dimensional simulation.

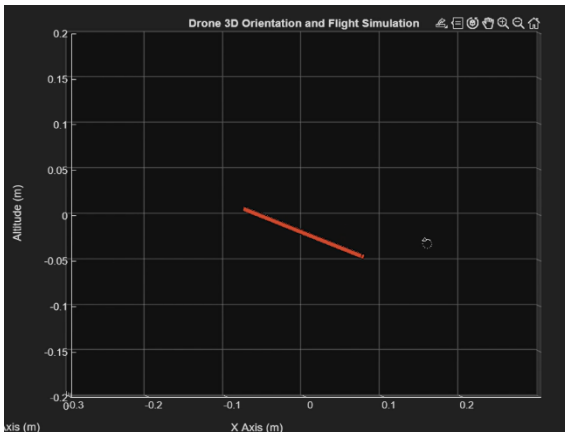


Figure 11. Pitch Down movement in the three-dimensional simulation.

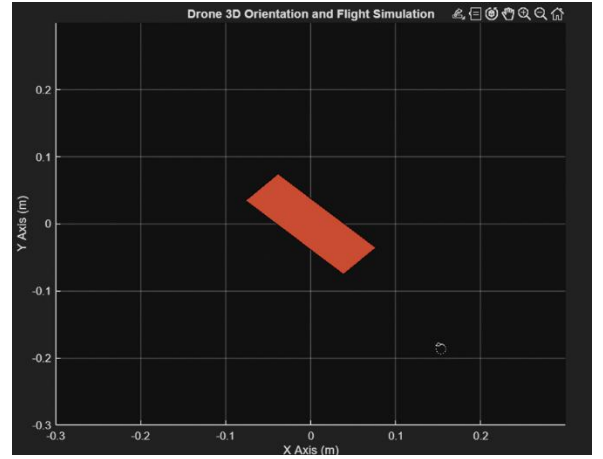


Figure 12. Clockwise yaw movement in the three-dimensional simulation.

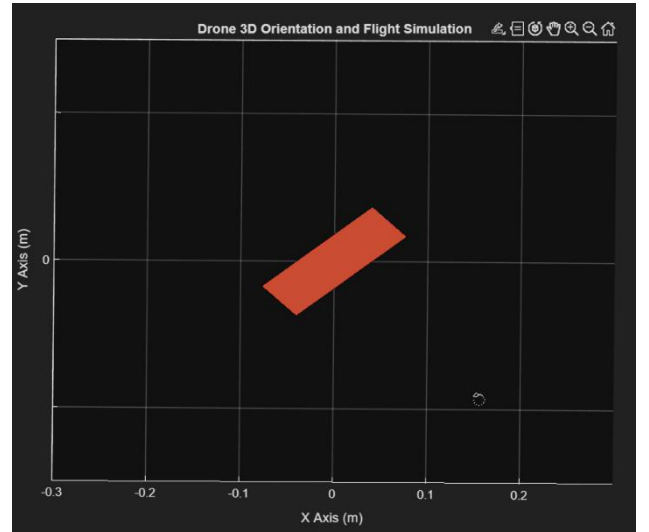


Figure 13. Counterclockwise yaw movement in the three-dimensional simulation.

VI. CONCLUSION

In this project, stable and low-noise state estimation was achieved from multi-sensor data by combining embedded systems and control theory. It was observed that hardware noise and systemic sensor errors can be overcome with a properly tuned Kalman filter and a serial communication-logging infrastructure. In addition, the system was made to operate more stably by adjusting the Kalman filter covariance values and eliminating hardware limitations (magnetic deviations, barometric noise, etc.). This comprehensive system architecture and filtering algorithms constitute a basis for autonomous control and data analysis studies in the field of Control and Automation Engineering.

REFERENCES

- "BMP280_STM32 Library", ciastkolog, GitHub Repository.
https://github.com/ciastkolog/BMP280_STM32/blob/master/BMP280/bmp280.h
- "BMP280_STM32 Implementation", ciastkolog, GitHub Repository.
https://github.com/ciastkolog/BMP280_STM32/blob/master/BMP280/bmp280.c
- D.Simon, "Optimal State Estimation: Kalman, H_infinity, and Nonlinear Approaches," John Wiley & Sons, Inc., 2006.
- Bosch Sensortec, "BMP280 Digital Pressure Sensor Data Sheet," BST-BMP280-DS001-26, Rev. 1.26, Oct. 2021.
- InvenSense, "MPU-6000 and MPU-6050 Product Specification," Document Number PS-MPU-6000A-00, Rev. 3.1, Oct. 2011.
- QST Corporation, "QMC5883P: 3-Axis Single Chip Magnetic Sensor Datasheet," Document Number 13-52-19, Rev. A, Shanghai, China.